

Predicting Contract Type and Duration Using Contract Clauses



BA 305: Business Decision-Making with Data
Boston University Questrom School of Business
Spring 2026

Vincent Sun, Xinjie Ji, Xinyi Wang

Introduction

Contracts are agreements between entities like companies and individuals often accompanied by a terms of service that decides what role and limitations each entity has within the agreement. Common examples of contracts in action can be seen in collaborations between your favorite athleisure brands like Nike and Michael Jordan to produce Jordan brand sneakers or even something close to home like the collaboration between Raising Cane's chicken fingers and singer Post Malone.

But for specialists who work with contracts on a daily basis, understanding the type of contract, the length of the typical contract, and how different clauses contained in the contract affect its identity, can be instrumental to accomplishing the task at hand. In roles like legal or auditorial services, workers often have to rely on reading each contract in order to discern the type of contract. This process is tedious and time consuming, however, key clauses inside of each contract can help create a profile of the type of contract, for example an Intellectual Property contract may have creative licensing terms and agreements as a part of its contract. Understanding these clauses are a great way to save time and effort when identifying the nature of contracts.

Our project will build on this topic. Our two questions will be to answer if we can use classification models to predict the **type** as well as the **duration** of different classes of contracts based on whether or not certain clauses exist inside of each contract. This model will seek to understand the relationship between clauses and contracts so it may potentially remedy the costly and tedious process of contract classification.

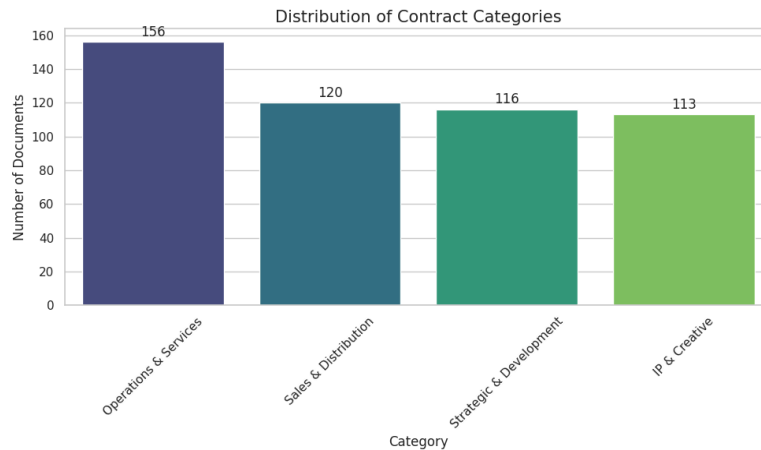
Data Overview

Our dataset was taken from a part of the Atticus project, containing a collection of 510 contracts that have been manually labeled under the supervision of experienced lawyers used mainly for contract analysis. Contained within each contract are ~40 different clauses, these clauses range between different topics, well known clauses are: exclusivity, non-compete, and IP ownership. The original dataset also contained 25 different types of contracts each with a boolean and text indicating if a specific clause exists and if so, the short text of the contents inside those clauses (Table 1, Appendix). The dataset and our analysis will all be using categorical labels to predict both contract class and contract duration.

In addition to boolean and text data, the dataset also contains columns with dates indicating when the contract first goes into effect and when the contract expires. The full contract contained additional data columns containing more specific information such as the entity names of the company/subjects which the contract affected as well as what geographic laws the contracts were subject to resulting in a pre-cleaned dataset size of 510 rows and 83 columns. However, for our two questions we mainly paid attention to the types of contract, the ~40 clauses, and the dates.

Data Cleaning and Feature Engineering

We started off by removing all unnecessary columns such as 'Parties','Parties-Answer' that we don't need for our questions. After that, we noticed the types of agreement are hidden within the Document name, such as 'VIDEO-ON-DEMAND CONTENT LICENSE



AGREEMENT' should be a Licence agreement, but it was not explicit marked in the data set, so we standardized the agreement type to a new column. However, there are insufficient samples per class, as well as a serious class imbalance, where there are 35 License Agreement, and only 8 Transportation Agreement. So we decided to group all the clauses into four groups, IP and Creative, Sales and Distribution, Operations and Services, and Strategic and Development (Figure 1), this

addressed class imbalance and reduced variances.

(Figure 1: Distribution of Contract Categories after grouping)

Predicting Contract Classification

To predict the types of contracts from the different input clauses and texts, in order to demonstrate model improvement and our choice of strategies, our group decided to use a two phase approach. First, we want to experiment with hyperparameter tuning to see if we could get sufficient predicting accuracy using **just** boolean responses to whether a clause exists or not (yes/no). Next, in order to perform more in-depth training and prediction, we employed the second phase, which included the text that also came with the booleans clauses. Lastly, we will evaluate each model and explain why we think these models failed/succeeded.

To begin, we used one hot encoding to turn our categorical columns such as booleans and duration into a form that the model can understand. We adopted the train and testing split of 80/20 for both models used in this prediction.

Logistic Regression with Booleans

To begin, we used logistic regression because of its ability to predict probabilities in classification problems. In log regression, a sigmoid activation function is used to map the probability of an observation belonging to one class or another within the 0-1 constraint and under the assumption that the total probability sums to 1 for all prediction classes. Logistic regression will then output the class with the highest probability given the input features.

For hyperparameter tuning, we defined five main performance tuners and set them to common learning values: *C regularization strength*, *Regularization Penalty*, *Solver*, *Class Weights*, and *Max_iter*

For the C regularization strength, our groups chose the default value of 1.0. C value indicates how much penalty the model is subjected to when learning the nuances contained in the dataset inputs. For example a small C-value like .1 means the model will be penalized more for each wrong classification, therefore when C is small, the model will tend to gravitate to more obvious surface level patterns to avoid large penalties. When C is large, the model is more likely to learn in-depth relationships because there is less penalty applied for wrong classifications. Next, we use L2 regularization to constrain the model so that it would not overly rely on any single input when predicting for class. This helps the model prioritize any signals that would have been missed due to being overshadowed by an input feature. For our gradient solver, we chose to use “lbfgs” instead of the standard Adam optimizer. We made this decision because lbfgs works better for smaller to medium datasets and for our dataset with 500 rows, we believe this decision best reflects our data dimensions. Next, we defined our class weights, we chose “balanced” because of our unbalanced duration column. Since we had much more long term entries than short term and medium terms as shown in (Figure 2, Appendix), by setting class weights to balanced, we ensure that the model is interpreting each duration class weights equally.

Lastly, we chose a maximum iteration of 2000, allowing us to train the model for 2000 steps, although we could have chosen a higher number of iterations, it doesn’t necessarily mean that the model will improve if it plateaus due to already extracting all of the boolean meanings.

Looking at our results in Table 1 of using just booleans to predict contract classes, we can see that even with extensive hyperparameter tuning, the model underperforms. Note the precision and recall for all classes hover around .5 except for strategic and development. This indicates that the model is struggling both to identify positive class values for all classes but also identifying true positives from all positive labels.

	precision	recall	f1-score	support
IP & Creative	0.50	0.52	0.51	21
Operations & Services	0.50	0.62	0.55	29
Sales & Distribution	0.45	0.43	0.44	21
Strategic & Development	0.43	0.29	0.34	21
accuracy			0.48	92
macro avg	0.47	0.46	0.46	92
weighted avg	0.47	0.48	0.47	92

(Table 1: Logistic regression performance results for boolean inputs and hyperparameter tuning)

Taking a look at (Figure 3, Appendix), we can see why that is. When we look at the predicted labels for strategic and development, almost half of the mispredicted labels for this class ended up in the sales and distribution class. This distribution is also reflected in the prediction for the other classes, we observed noise around the sides of the diagonal, showcasing that the true value is being mispredicted by our model at a high level. Overall this tells us that with the current hyperparameters and boolean input values, our model is unable to effectively distinguish one class from another due to noise such as overlapping clauses belonging to different classes.

Logistic Regression TFIDF

To tackle this problem, our group decided to use TF-IDF to identify textual terms and frequency for the input data and see if by incorporating textual data, we can increase the prediction accuracy for each class. TF-IDF stands for term frequency inverse document frequency. TF-IDF works by looking at each text and assigning weights to each word that

appears in the text based on how many times the word appears in the text and how little it appears in texts from other classes. This helps TF-IDF assign word importance to class specific words. The main data preprocessing change we did in this section was aggregating all of the texts from clauses which had a “yes” for if the clauses existed, taking their accompanying text clause and aggregating it into a format that TF-IDF can understand and process.

Key parameters in TF-IDF are: *Max_features*, *Min_df*, *Max_df*, *Custom_weights*.

Max_features tells us how large of a vocabulary do we want TF-IDF to access, we picked the value 500 to account for 500 of the most common words as for each prediction to base the model output off of. *Min_df* tells the TF-IDF model the minimum times a word needs to appear in the text for it to be considered as a text feature, we picked 2 because for extremely specialized words like a formal name or entity gives no predictive power or learning to our model and we should ignore it as it will just become noise. *Max_df* tells us the maximum document frequency, this tells us how many times a word can appear in all documents before it becomes noise, for our model we chose .7, such that if a word appears in 70% of the texts from all classes, then it is most likely a filler word and offers no class insights to the TF-IDF model. Lastly, since our previous model struggled with strategic and development classification, we chose to manually adjust our class weights to be more sensitive to strategic and development, making mistakes in this class become more severe than mistakes in other classes.

By using these hyperparameters in TF-IDF, we observe the results shown in Table 2. Compared to not using texts at all as shown in Table 1 logistic regression, we can see that both precision and recall improve by a noticeable amount in some categories, similarly the f1-score also saw improvement in both the operations and strategic contract classes. However, we also noticed that by placing a higher sensitivity to mistakes on strategies and development, our recall score for that class shot straight up but our precision suffered. This is a classic case of imbalance as the model is picky when choosing what to classify as strategy and development, and for our goal of assigning diverse contract classes, a low precision with high recall would mean there needs to be much more manual review needed since only the most cookie cutter examples would be correctly classified.

	precision	recall	f1-score
IP & Creative	0.600000	0.428571	0.500000
Operations & Services	0.789474	0.517241	0.625000
Sales & Distribution	0.583333	0.333333	0.424242
Strategic & Development	0.369565	0.809524	0.507463

(Table 2, TF-IDF performance results for manually adjusted hyperparameters and document frequency)

To address this issue and improve the performance of TF-IDF, our group also thought of removing common stop words that appear in **all** texts for all classes. These words are effectively noise that are found in most clauses and serve no purpose other than act as padding. For example, (Figure 4, Appendix) lists potential stop words, words, some words evaluated and removed were: shall, agreement, section, contract, etc.

In addition to removing stop words, we also seek to fix the issue with precision and recall imbalance by optimizing our f1-score through finding the best contract classification threshold. Our group decided to experiment with different hyperparameters for our TF-IDF logistic regression model. We started by picking an array of hyperparameters and looping through each plausible value combination. For example, we chose to better optimize for the sensitivity to each class error by iterating through each error weight *[1.0, 2.0, 3.0, 4.0, 5.0]*, with 1.0 being least sensitive to errors and 5.0 being most sensitive to errors. We also used different C regularization strength values to balance data exploration to better identify nuances in the text. Lastly we tested

the most optimal number of word/features for the TF-IDF model to remember in order to correctly classify the contract class. After running the optimization loop with filtered stop words, we achieved a top f1-score of .628, our best hyperparameter combination were:

'C': 7,
'max_features': 500,
weights': {'Operations & Services': 1.0, 'Strategic & Development': 2.0, 'Sales & Distribution': 1.0, 'IP & Creative': 2.0}

Taking a look at the results of the TF-idf model with these hyperparameters as inputs, we get a final TD-IDF logistic model performance (Table 3):

	precision	recall	f1-score
IP & Creative	0.678571	0.904762	0.775510
Operations & Services	0.714286	0.517241	0.600000
Sales & Distribution	0.687500	0.523810	0.594595
Strategic & Development	0.481481	0.619048	0.541667

(Table 3, TF-IDF performance metrics for optimized hyperparameters and manual reviewed stop words)

Logistic regression and TF-IDF Model Evaluation

Taking everything into account, we can see that while TF-IDF with text included helped steer the logistic regression model in the right direction, the model is still not perfect. In Table 3, we can see that our resulting metrics compared to Table 1 and Table 2 were much better and more balanced between precision and recall. This tells us that our hyperparameter optimization helped balance precision and recall. However, this also means that the extent of our logistic regression and TF-IDF is mostly constrained within the way the model predicts and not because of our tuning. Through extensive modeling with TF-IDF we believe that the main reason why the model is not performing as well as we would have liked can be attributed to the nature of TF-IDF. Since TF-IDF only looks at the number of times a particular word appears in the document and assigns weights based on that number, the model doesn't consider what each word actually contributes to the document at all. So if a word carries more context towards the beginning of a sentence than towards the end of a sentence, TF-IDF does not capture semantic importance of the word, leading to loss of predictive power. This is reflected in (Figure 5, Appendix), we see that the model struggles to classify strategic and development contract class, to fix this we will look into ways we can mediate this through trying to understand the semantic meaning of texts.

Neural Network with Booleans

Taking what we learned from performing logistic regression, our group felt pursuing Neural Networks was the next step in order to learn more about the intricacies inside of each clause. Since Neural networks act as a black box with configurable learning layers and hyper parameters it is a logical model choice when it comes to predicting the countless decisions inside of our data that a common decision tree cannot track.

For hyper parameter tuning, we mainly focused on: *Size of hidden layer, alpha, and activation.*

In neural networks, the size of the hidden layer decides how much learning is done by each layer and subsequent layers. A high number of layers means potentially more learning but conversely more at risk of overfitting. Alpha values tell the model how far to step in a direction

in order to reach the optimal global minimum for gradient descent, a high alpha value may overshoot the optimal answer but a small alpha value may never reach the minima at all. We also experimented with two types of activation functions, one Relu and one Tanh, both of which are plausible and help with ensuring the neural network fires when sufficient cues are present. Additionally, we also used the industry standard Adam solver and incorporated early stopping, which ensured our model did not waste resources if the model performance failed to improve in later iterations.

For phase 1 using just the boolean responses to predict contract classes, we used a similar hyperparameter optimization approach where we iterated through each combination of hyperparameters and selected whichever combination achieved the highest f1-score. Through this we achieved the optimal hyperparameters listed below with a f1-score of .43:

'layers': (100,),
'alpha': 0.001,
'activation': 'tanh'

	precision	recall	f1-score
IP & Creative	0.600000	0.428571	0.500000
Operations & Services	0.437500	0.724138	0.545455
Sales & Distribution	0.437500	0.333333	0.378378
Strategic & Development	0.384615	0.238095	0.294118

(Table 4: Neural network performance metrics with hyperparameter tuning on boolean clauses)

Breaking down these results, we can see that the neural network model trained on only the boolean clause inputs had subpar performance. Examining (Figure 6, Appendix), it is clear that the model exhibit similar results as the logistic regression model trained on only boolean inputs, both sales & distribution and strategic & development performed worse than other classes, this reenforces our prior claim that strictly evaluating the boolean answers when predicting contract class is insufficient even across prediction models.

Neural Network with Hyperparameter Tuning on Boolean and Text

Taking what we learned, we sought to incorporate both the boolean and text data in the neural network as well as increase the number of combinations for the neural networks layers in our hyperparameter optimization. Running the optimization based on the highest f1-score, and keeping all of the hyperparameters the same, for this model, we found the best hyperparameters with a average f1-score of .61 was:

'layers': (64, 64, 64),
'alpha': 0.01,
'activation': 'relu'

	precision	recall	f1-score
IP & Creative	0.619048	0.619048	0.619048
Operations & Services	0.740741	0.689655	0.714286
Sales & Distribution	0.473684	0.428571	0.450000
Strategic & Development	0.600000	0.714286	0.652174

(Table 5: Neural network performance metrics with hyperparameter tuning boolean and text)

Looking at Table 5 results and visualizing the results in (Figure 7, Appendix), we get a much cleaner confusion matrix than previously using only the boolean inputs as shown by a cleaner diagonal line between the x and y axis.

Interpreting these results, we can see that much of the noise is filtered out by incorporating textual embeddings inside of the neural network. Additionally, because we

included more complex data in the form of text, our hyperparameters also changed to reflect this complexity. In the number of layers for the boolean only model we used 1 hidden layer with 100 prediction nodes, but in the current model we used 3 hidden layers with 64 nodes. We interpreted the current model as a deep neural network since it has multiple interconnected hidden layers. Unlike the boolean neural network where 1 hidden layer is enough to understand the yes or no relationship of each contract and its clauses, our current 3 hidden layer model makes sense since the model is trying to learn about the grammatical nuances for each word in the text and how different words interact with each other to determine the contract class.

Despite this, we still weren't satisfied with the performance of this model, although neural networks are effective in learning the interconnected relationships between words, it's still inefficient. Everytime we pass in the input text data, we initialize a new neural network that learns the nuances of the text from scratch. This means a portion of the mispredicted results can be attributed to surface level semantic meaning and deeper contextual meaning slip under our model's radar.

Neural Network + BERT Encoding

Since we already know that neural networks worked the best to understand the semantic meaning in textual data to classify contracts, one way our group found that can improve the result is to use a pretrained text model in our neural network. BERT is a pretrained text model including training from millions of text sources like Wikipedia. Using BERT in our neural network architecture would help lighten the training load on the neural network and enable our model to focus more learning resources on predicting rather than understanding semantic meaning of texts.

In addition to incorporating transfer learning, our group also added in an additional pooling parameter. In language learning, pooling looks at all the features and selects only the most important features in a group, this helps the model with denoising when identifying what to keep and what to ignore.

Using an optimization loop for our hyperparameters as we did many times before, we get the best average f1-score we have seen thus far of $\sim .74$ with the hyperparameters:

```
'pooling': 'max',
'alpha': 0.01,
'layers': (100,)
```

Interpreting these hyperparameters, one unique observation we made in regards to these hyperparameters was that contrary to the previous neural network model using text embeddings, the number of hidden layers dropped down to only one hidden layer with 100 nodes. We interpreted this as a result of using BERT since by using a pretrained text model, our model does not require a deep neural network to learn all of the word nuances.

	precision	recall	f1-score
IP & Creative	0.772727	0.809524	0.790698
Operations & Services	0.777778	0.724138	0.750000
Sales & Distribution	0.800000	0.761905	0.780488
Strategic & Development	0.608696	0.666667	0.636364

(Table 6: Performance metrics for Neural network + BERT encoding results)

As shown in (Figure 8, Appendix) and its evaluation statistic Table 6, our group's final neural network with BERT model's results were much more uniform and accurate compared to the previous models we have tested so far. From this model, we can see that most of our

f1-scores for most classes hover around .8 with the strategic & development class receiving the lowest f1-score. There are a couple of theories of why this is the case. One reason is that perhaps the BERT model was not explicitly trained on the terms found in strategic and development contracts, another reason could be because of our data and how the strategic & development class was defined. We believe the contracts originally assigned to this class may not have been as uniform as we would have anticipated and that there was just more variability in the textual nature of the clauses in this category. Hence, there may have been more words or phrases in this class that were on the fence with another class, stumping the model. Despite this, our group was extremely satisfied with our model outputs, although not perfect, they did indicate to us that predicting the types of contract based on accompanying clauses is possible even with the limited sample we were given.

Predicting Contract Duration

a. Target Feature Engineering: Computing Contract Duration

The dataset does not include a pre-computed duration column. In order to predict the duration, we need to know the exact days that each contract is legitimate. In our dataset, it carries two date fields: Agreement Date-Answer (the effective date) and Expiration Date-Answer (the end date), both stored as strings in MM/DD/YY format. We converted these to datetime objects using `pd.to_datetime()` with `format='%m/%d/%y'`, then computed:

$$\text{duration_days} = \text{Expiration Date-Answer} - \text{Agreement Date-Answer}$$

For the rows that have ‘perpetual’ as expiration date answer, we substitute the duration with 10,000 days as a proxy and convert everything to numeric.

b. Duration Binning and Label Encoding

We binned `duration_days` into four categories for classification prediction:

Short-term: under 5 years (< 1825 days)

Medium-term: 5 to 20 years (1825 – 7300 days)

Long-term: 20 years to 10000 days (7300 – 10000 days)

Perpetual: greater than or equal to 10000 days, as we defined 10000 as ‘perpetual’

After defining the duration bins, we use label encoding to encode each duration category for the models to run on it (Table 7, Appendix).

Why regression doesn’t work

To predict the exact contract duration in days, we ran three regression models: Random Forest Regressor (200 trees, max depth 25), Ridge Regression ($\alpha = 10.0$ on standardized features), and an MLP Neural Network (hidden layers 100 and 50, L2 $\alpha = 0.01$). Perpetual contracts were assigned a proxy value of 10000 days, and remaining missing values were filled with the median. All three models were evaluated on an 80/20 split using RMSE, MAE, and R^2 .

Model	RMSE (days)	R^2	MAE (days)
Random Forest	4548.718611	-0.123415	3960.524469
Regression	4240.845999	0.023512	3784.189715
Neural Network	4360.936815	-0.032575	3890.569475

(Table 8: Regression results evaluation for each model tested)

The results were poor as shown in Table 8. R^2 values were near zero or negative across all three models, meaning they were essentially no better than predicting the dataset mean every time. RMSE was huge for all three models and ran into the thousands of days.

The failure makes sense because boolean clause indicators tell you whether the clauses are present, but a contract with an Exclusivity clause could run for one year or twenty. Clause presence just doesn't predict duration length in a way that regression could measure with good performance.

The target variable, the duration, is also an issue. When we substitute perpetual with 10000 days and convert everything to numeric, the resulting distribution is heavily skewed, where the median sits at 10000 days (27.4 years) and the mean at 6,189 days (17.0 years). Moreover, 365 days and 10000 days (perpetual) represent fundamentally different contractual structures. Asking the model to learn the gradient between them was the wrong approach. On top of that, 54% of the dataset is perpetual, which regression treats as 10000 days, which made the scale heavily skewed to longer days and failed to reflect a fair prediction on the duration of shorter contract in the test set. Hence, it seemed that reframing the problem as classification could be a better way to predict duration.

Classification with Boolean Features Only

Using the 32 binary clause features and all 505 contracts, we tested two classifiers on an 80/20 stratified split.

Random Forest was first run with default settings (200 estimators, max depth 25), then tuned via GridSearchCV over *n_estimators*, *max_depth*, *min_samples_split*, and *class_weight* with 5-fold cross-validation. The best configuration is: *class_weight='balanced_subsample'*, *max_depth=25*, *min_samples_split=2*, *n_estimators=100*, which improved accuracy from 52.5% to 58.4%.

Logistic Regression was similarly tuned, with the best parameters found to be *C=0.1*, *penalty='l1'*, *solver='saga'*. The tuned model achieved 59.4% accuracy.

RF (baseline)52.5%	RF (tuned)58.4%
LR (baseline)57.4%	LR (tuned)59.4%

Random Forest Classification Report:					Logistic Regression Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Long-term	0.00	0.00	0.00	1	Long-term	0.00	0.00	0.00	1
Medium-term	0.00	0.00	0.00	13	Medium-term	0.00	0.00	0.00	13
Perpetual	0.65	0.86	0.74	56	Perpetual	0.60	0.89	0.71	56
Short-term	0.46	0.35	0.40	31	Short-term	0.59	0.32	0.42	31
accuracy			0.58	101	accuracy			0.59	101
macro avg	0.28	0.30	0.28	101	macro avg	0.30	0.30	0.28	101
weighted avg	0.50	0.58	0.53	101	weighted avg	0.51	0.59	0.52	101

Random Forest Confusion Matrix:

(Table 9 (left): Random Forest evaluation with booleans)

(Table 10 (right): Logistic Regression evaluation with booleans)

Looking at the classification reports in Table 9 and Table 10 and confusion matrix (Figure 9 & Figure 10, Appendix), both models are predicting Perpetual and Short-term only. Perpetual gets strong recall (RF: 86%, LR: 89%) simply because it dominates the dataset. Medium-term gets zero precision and recall from both models because it only has 13 test samples and no strong clause-level signal separating it from Perpetual or Short-term. Long-term (1 test sample) is

similarly invisible. Therefore, the overall accuracy numbers are somewhat misleading given this behavior.

To address this issue, we tried multiple binning configurations such as divide non-perpetual contracts into three equal thirds at the 33rd percentile (~536 days / 1.5 yr) and 67th (~1778 days / 4.9 yr), but all configurations hit the ceiling because the boolean features carry too little discriminative signal. The deeper reason none of these work well comes from the feature importance analysis. The top features– Anti-Assignment-Answer, Termination For Convenience-Answer, Insurance-Answer, Revenue/Profit Sharing-Answer–each have feature importance values between 0.035 and 0.056. They're spread nearly flat across all 32 features with no single dominant signal to tell whether they are significant for a duration bucket. The binary clauses do not contain enough information that actually determines duration. This motivates incorporating TF-IDF because we think that the actual clause language should contain duration-relevant information that booleans cannot capture.

Classification with TF-IDF

In the TF-IDF preprocessing stage, we conducted the similar setup for the previous question with the engineering of combined_text column, the custom stop words from the top 10 most frequent words appearing in the text, and the refined TfidfVectorizer configured with:

```
max_features=1000
ngram_range=(1,2)
min_df=2, max_df=0.7
stop_words=custom_stop_words
```

Custom class weights were set to: *Short-term: 1, Medium-term: 2, Long-term: 3, Perpetual: 3* for the models to perform better on the minority classes.

After running TF-IDF on both models, the performances for both models are shown below:

	precision	recall	f1-score	support		precision	recall	f1-score	support
Long-term	0.00	0.00	0.00	1	Long-term	0.00	0.00	0.00	1
Medium-term	0.00	0.00	0.00	13	Medium-term	0.33	0.15	0.21	13
Perpetual	1.00	0.45	0.62	11	Perpetual	0.47	0.73	0.57	11
Short-term	0.61	1.00	0.76	31	Short-term	0.67	0.71	0.69	31
accuracy			0.64	56	accuracy			0.57	56
macro avg	0.40	0.36	0.35	56	macro avg	0.37	0.40	0.37	56
weighted avg	0.53	0.64	0.54	56	weighted avg	0.54	0.57	0.54	56

(Table 11 (left): Random Forest evaluation with TF-IDF)

(Table 12 (right): Logistic Regression evaluation with TF-IDF)

From the classification reports in Table 11 and Table 12 and the confusion matrices (Figure 11 & Figure 12, Appendix), there is an improvement in the accuracy for both models, with random forest model reaching 64% and logistic regression model reaching 57%. Moreover, in the random forest model, there's a 100% recall on short term contract, meaning that it makes no mistake in predicting the short term category. However, we conclude that logistic regression benefits more from TF-IDF and is the superior model. Although logistic regression + TF-IDF has a lower accuracy compared to random forest + TF-IDF, even lower than without applying TF-IDF, we have seen an improvement in the predicting ability for medium term and perpetual. With LR + TF-IDF, medium term category achieves F1 of 0.21, compared to 0.00 across all

boolean-only configurations for both models. Perpetual also improves substantially under LR + TF-IDF, going from near-zero to F1 of 0.50. Perhaps the clause text provides signals that boolean features cannot, such as terms like “irrevocable” or “in perpetuity” in the clause language help separate perpetual from the other duration categories. Moreover, to think conceptually why logistic regression benefits more from the TF-IDF and performs well on a more diverse category compared to random forests that solely do well on predicting short term contracts, we can think about how the models classify text. Logistic regression works by assigning a direct weight to each term for each class, so when a word like “initial term” consistently appears in Short-term contracts, the model learns that association. Random forest, on the other hand, builds hundreds of decision trees, each node randomly sampling a subset of features to split on. When the feature space suddenly expands to over a large amount of TF-IDF text columns on top of the 32 boolean ones, most trees end up splitting on uninformative terms by chance, and the signal from the few useful text features gets buried or ignored in the final predicting pattern that random forest learns. Hence, the result is that RF + TF-IDF defaulted to predicting Short-term almost every time, and it missed other categories entirely despite the class weighting we applied.

Conclusion

To conclude, based on our results from both research questions, we would recommend using neural networks to classify contract classes. Neural networks are versatile and alongside transfer learning models, capture many of the nuances contained in contract clauses. For the duration prediction, we frame it as a classification task and discover that logistic regression with TF-IDF is the strongest configuration since the model extracts more information in the text and clauses that are used to identify the specific length category they belong to. However, due to a limitation in the data with only 500+ contracts to extract meaning from a highly skewed target feature towards perpetual, we acknowledge the possibility of overfitting and model’s performance of predicting certain categories better than the others and a ceiling around 60% accuracy. Breaking this ceiling would likely require more advanced tuning, more targeted features of duration from contract text, or more data points beyond what CUAD provides. Moreover, the bin for different year limits for the duration categories should be more refined and detailed given more legal and business context. These may serve as a topic for future exploration.

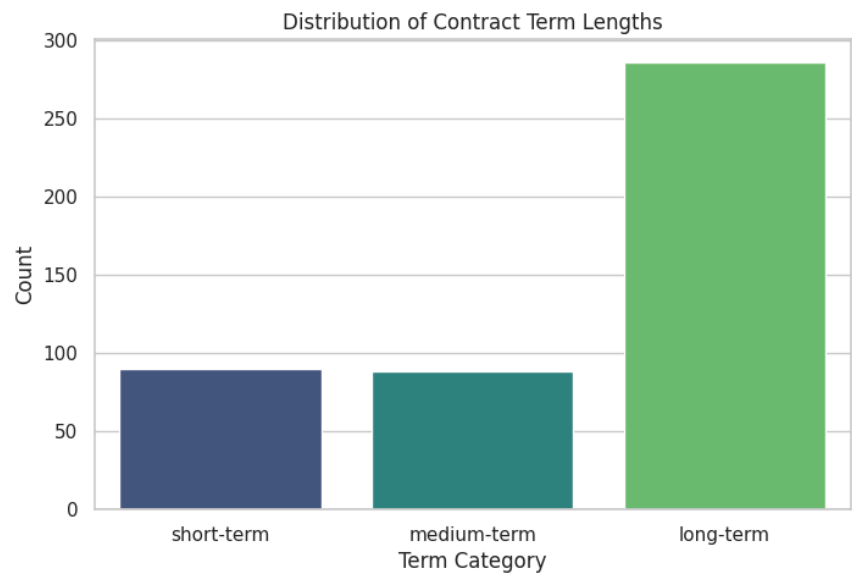
Appendix

Full Value Counts:

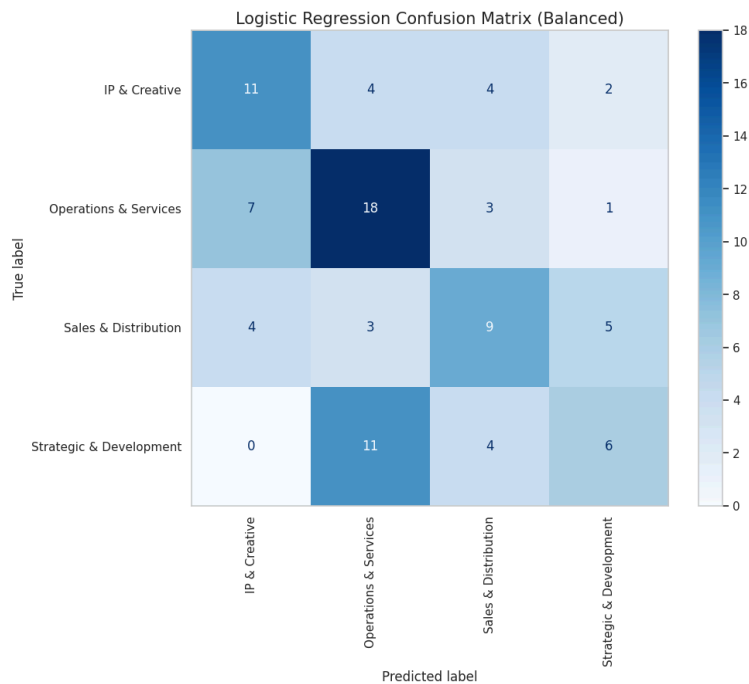
agreement-type	
Other	44
License Agreement	35
Development Agreement	33
Sponsorship Agreement	32
Strategic Alliance Agreement	31
Distributor Agreement	30
Service Agreement	29
Maintenance Agreement	27
Co-Branding Agreement	23
IP Agreement	23
Endorsement Agreement	22
Supply Agreement	17
Manufacturing Agreement	17
Outsourcing Agreement	16
Hosting Agreement	16
Marketing Agreement	15
Franchise Agreement	13
Agency Agreement	13
Collaboration Agreement	12
Consulting Agreement	12
Promotion Agreement	12
Affiliate Agreement	10
Reseller Agreement	10
Joint Venture Agreement	10
Transportation Agreement	8

Name: count, dtype: int64

(Table 1: 25 types of contracts original not grouped)



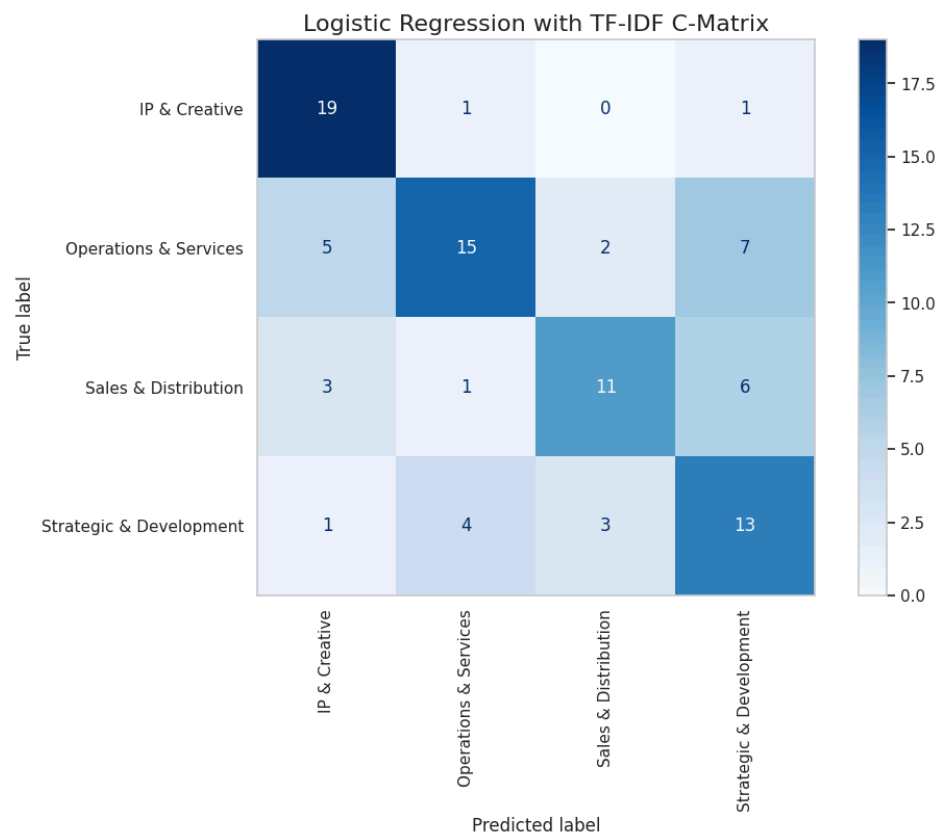
(Figure 2: Distribution of contract term lengths)



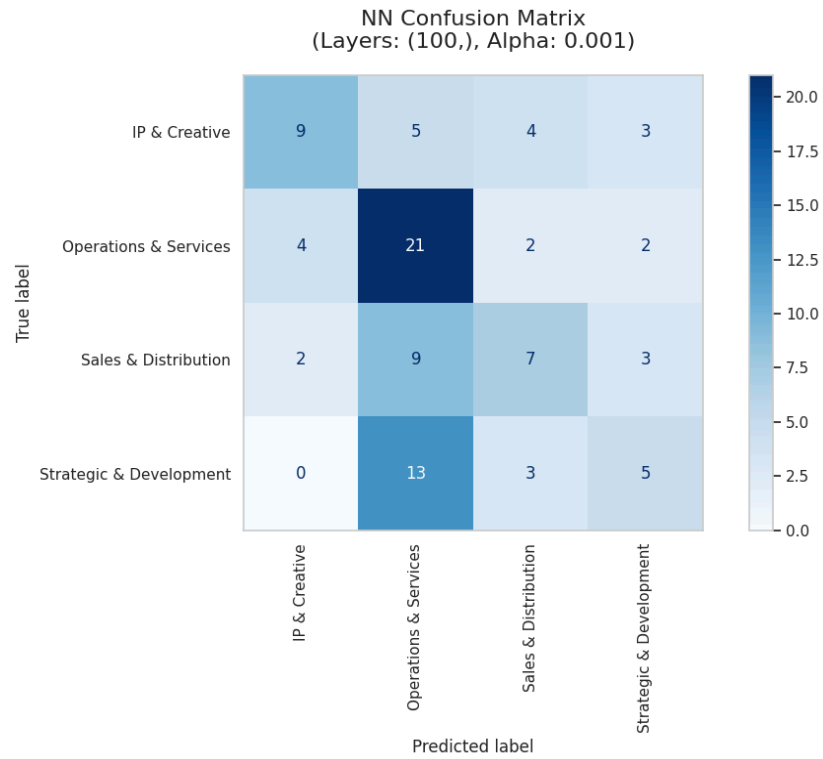
(Figure 3: Logistic regression confusion matrix for boolean inputs and hyperparameter tuning)

	word	count
6311	shall	5427
544	agreement	5045
5052	party	3870
6216	section	1991
7282	use	1837
1498	company	1806
6064	right	1769
5428	product	1690
4046	license	1608
5430	products	1506
6066	rights	1476
4704	non	1425
2760	exclusive	1317
5518	provided	1250
6883	term	1234
7577	written	1229
3533	including	1215
1133	business	1196
5123	period	1092
5394	prior	1089

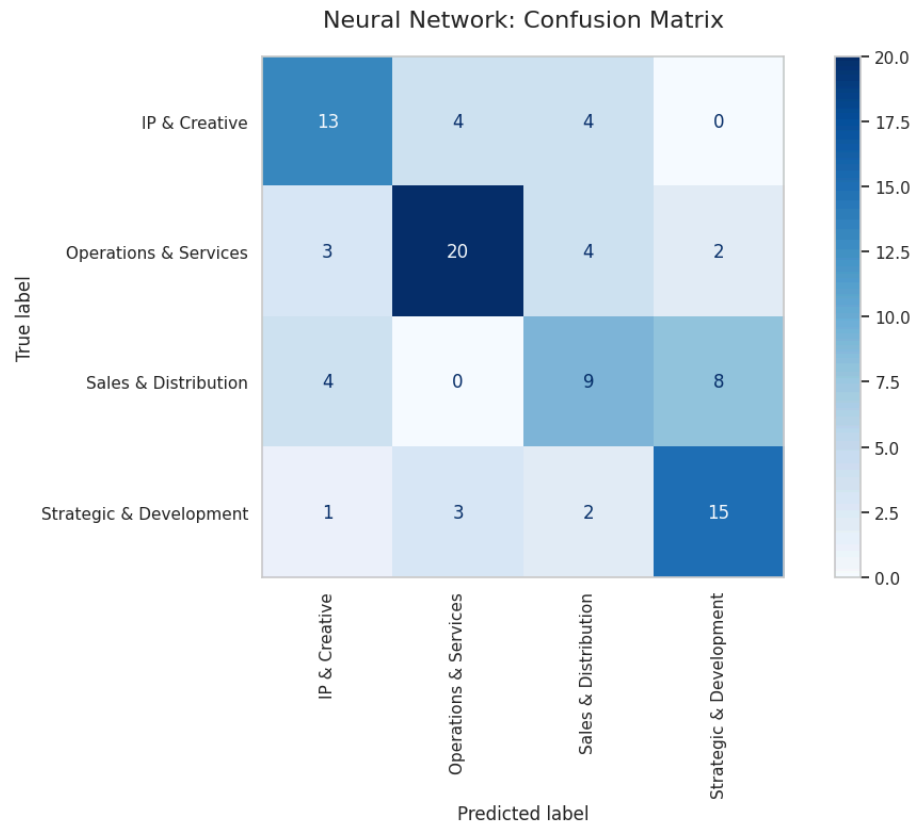
(Figure 4: stop words for TF-IDF)



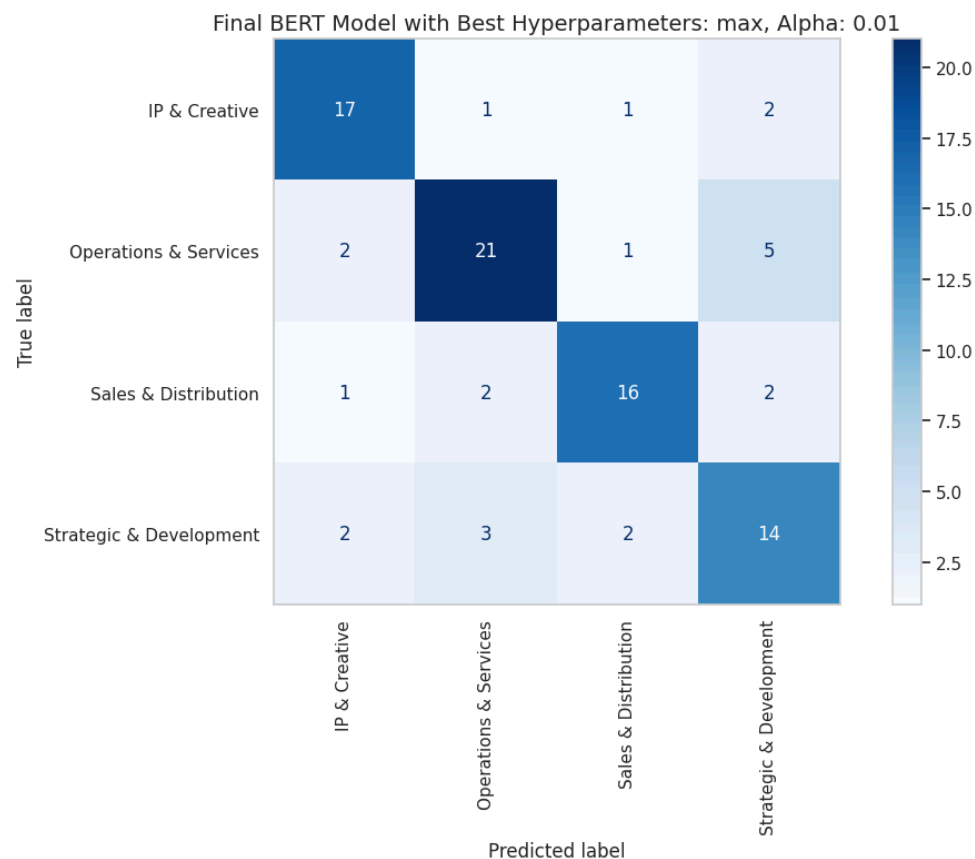
(Figure 5: TF-IDF confusion matrix output for optimized hyperparameters and manual reviewed stop words)



(Figure 6: Neural network confusion matrix with hyperparameter tuning on boolean clauses)



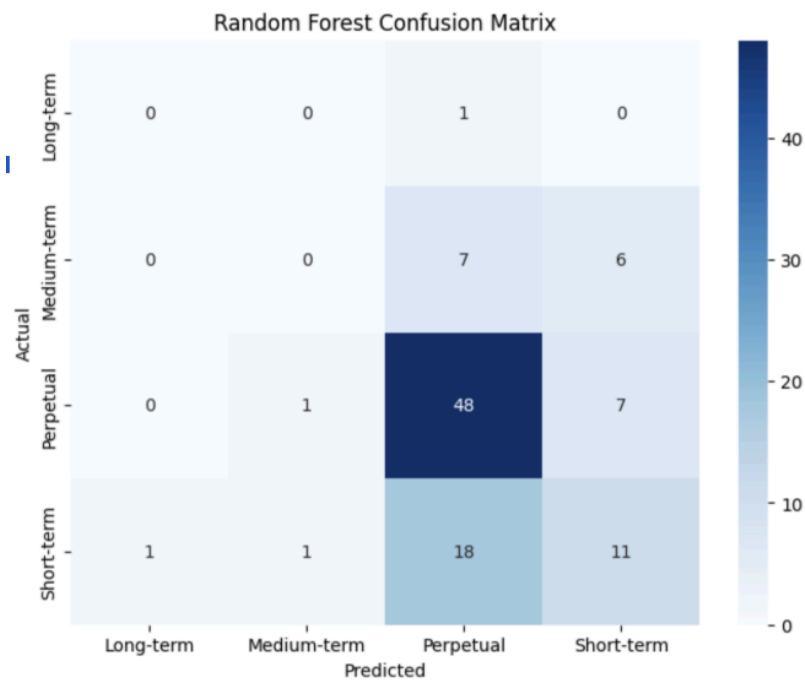
(Figure 7: Neural network confusion matrix with hyperparameter tuning boolean and text)



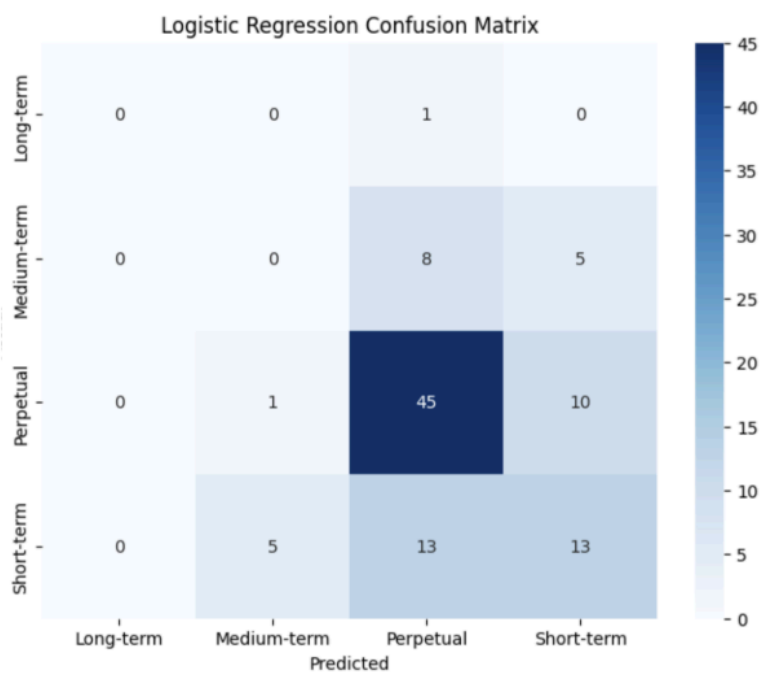
(Figure 8: Confusion matrix for neural network + BERT encoding results)

Label Mapping:
0: Long-term
1: Medium-term
2: Perpetual
3: Short-term

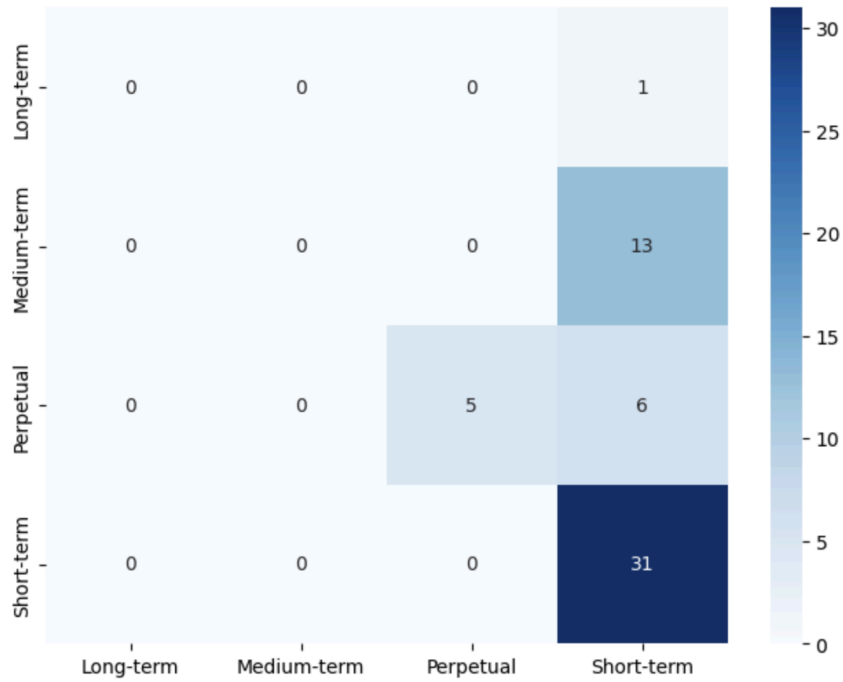
(Table 7: LabelEncoder result for label mapping for each category)



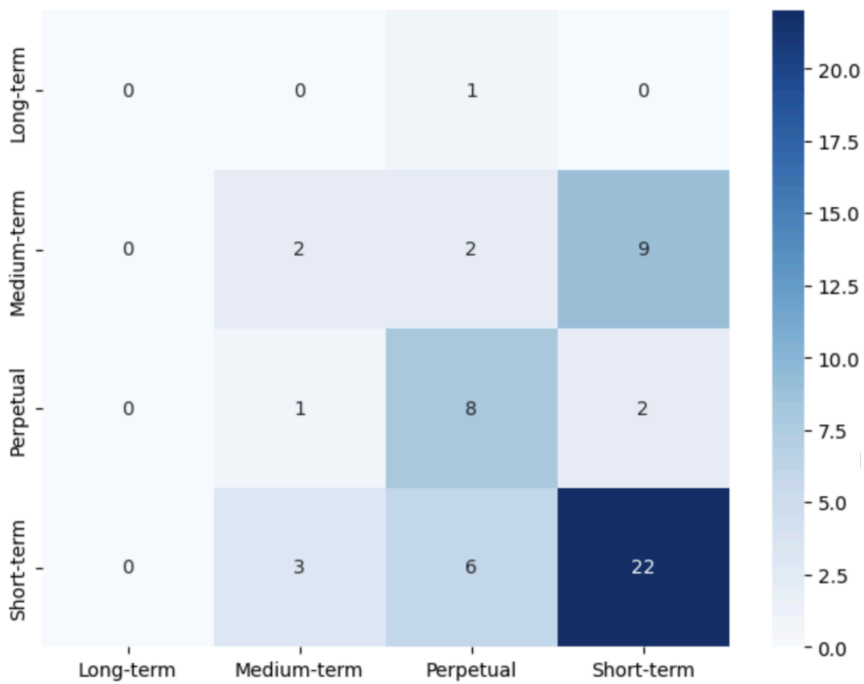
(Figure 9: Random forest evaluation with booleans)



(Figure 10: Random forest evaluation with booleans)



(Figure 11: Random forest evaluation with TF-IDF)



(Figure 12: Logistic regression evaluation with TF-IDF)